

LogPress

로그 특화 KV Cache 압축 for LLM RCA

2026 · W32 | 08.04(화) – 08.09(일)



진행 요약

Tokenizer · Model Forward · KV Cache · kvpress 환경 준비



KVPress 코드 리뷰

BasePress / ScorerPress · Line↔Token Mapping · Pipeline · 백업 경로



주간 진행 보고

킬스위치 M1 GO · Nezha 2층 데이터 착수 · 다음 액션

PART A

진행 요약

LogPress Week 3 · Baseline 구현을 위한 기본 흐름 확인

이번 주에 한 일

Baseline 구현을 위한 기본 흐름 확인

기능 완성이라기보다, 전체 파이프라인이 도는지 확인한 주

완료한 것

완료

- Tokenizer 출력 구조 확인 (input_ids, attention_mask)
- 모델 forward 실행 및 outputs 구조 확인
- past_key_values 에서 layer 0 Key/Value shape 확인
- kvpress 용 가상환경 분리 · StreamingLLMPress import 확인

확인한 결과

KV shape가 입력 토큰 길이에 따라 변한다

짧은 입력과 여러 줄 로그 입력에서 KV shape 변화를 실측

실습 환경

- Practice01.py · streamingllm_smoke.py
- Llama tokenizer/model 실습
- Qwen2.5-3B-Instruct 사용
- DynamicCache 타입 확인 · layer 0 K/V shape 확인

Full KV shape 실측

짧은 입력

[1, 2, 4, 128]

여러 줄 로그

[1, 2, 131, 128]

seq_len 축이 입력 길이에 따라 4 → 131로 증가

현재 상태와 한계

아직 압축 결과·보존율을 주장할 단계는 아니다

현재까지 완료

완료

- Full KV shape 확인까지 완료
- StreamingLLMPress import 가능 상태 확인
- 여러 줄 로그 입력으로 seq_len 131 까지 확인

아직 미완료

예정

- StreamingLLMPress 실제 적용
- 압축 전후 KV shape 비교
- Thunderbird 최종 1시간 윈도우 평가
- 이상 로그 보존율 계산

다음 작업

StreamingLLM 압축이 실제로 KV를 줄이는지 검증

다음 주 실행

- StreamingLLMPress(compression_ratio=0.5) 적용
- Full KV 와 압축 KV shape 비교
- 실제 Thunderbird message 기반 입력으로 확장
- 결과 저장 형식 정리

이번 목표

“압축이 실제로 KV seq_len 을 줄이는지”를 먼저 검증

PART C

KVPress 코드 리뷰

Transformer/Token · KV Cache · KVPress 구조 분석 및 LogPress 연결

이번 주 진행 요약

KVPress가 어디서 어떻게 압축되는지 분석

겹치는 내용을 묶어 핵심 진행 사항만 정리

Transformer / Token 구조

- Token이 모델 내부에서 어떻게 표현되는지 학습
- Attention · KV Cache 까지의 구조
- KV 압축에서 token 위치가 중요한 이유 이해

KVPress 코드 리뷰

- BasePress / ScorerPress 역할 분석
- 기존 Press 들의 토큰 선택 기준 비교
- 실제 KV Cache 압축 흐름 파악

프로젝트 연결

- 로그 line 과 token 을 연결하는 방법 실습
- Pipeline 코드 리뷰 진행
- Cache 직접 제어 백업 경로도 함께 분석

1. TOKEN 구조 변화와 KV CACHE 이해

Token ID → Embedding → Q/K/V → Attention → KV Cache

Token ID → Embedding → Head별 표현 → Q / K / V → Attention → KV Cache

토큰 구조 변화

- 입력 token 은 처음엔 정수 ID 형태
- Embedding 이후 각 token 이 벡터 표현으로 변환
- Attention 에서 여러 head 로 나뉘어 서로 다른 관계 학습
- Query/Key/Value 역할로 문맥 정보를 계산

KV Cache 와 압축

- 이후 생성에서 다시 쓸 Key/Value 를 Cache 에 저장
- 긴 입력일수록 저장되는 token 수가 계속 증가
- KVPress 는 벡터를 줄이는 게 아니라 **불필요한 token 위치를 제거**
- 따라서 '어떤 token 을 남길 것인가'가 핵심

2. KVPress 코드 리뷰

구성 요소가 담당하는 역할 중심으로 분석

BasePress

- 모델 Attention 층에 압축 기능을 연결
- 현재 층 KV Cache 를 가져옴
- 압축된 Cache 를 다시 모델에 저장
- → 압축이 실행되는 전체 틀

ScorerPress

- 토큰별 중요도를 이용하는 공통 구조
- 중요한 token 을 선택 제거
- 선택된 token 의 Key/Value 만 유지
- → 점수 기반 실제 압축 담당

위치 기반
방식

- 처음·최근 token 을 우선 보존
- 중간 token 을 주로 제거
- 단순한 baseline 으로 활용

Attention
/ Layer 기
반

- 모델이 많이 참고한 token 보존
- 또는 layer 마다 압축 강도를 다르게
- 중요도 계산 방식만 달라짐

공통점 · 중요도를 판단 → 남길 token 을 선택 → 해당 Key/Value 만 유지

3. LOG LINE-TOKEN MAPPING 과 LOGPRESS 연결

원본 로그 → Tokenizer → 원문 위치 확인 → Line↔Token → 중요도 → KV 압축

Mapping 실습

- INFO/WARN/ERROR 로그가 token 으로 어떻게 분리되는지 확인
- 각 token 이 원문의 어느 문자 위치에서 왔는지 추적
- UUID·IP·파일 경로처럼 여러 token 으로 나뉘는 경우도 확인
- 한 로그 line 에 속하는 token 목록을 만들 수 있음을 확인

LogPress 연결 방향

- 먼저 로그 line 단위로 중요도를 판단
- 중요한 line 에 해당하는 token 을 찾아 중요도를 전달
- 기존 KVPress 의 token 선택/압축 구조를 재사용
- 로그 특화 신호가 KV 압축에 도움 되는지 비교 가능

4. PIPELINE 코드 리뷰 · CACHE 백업 경로 분석

본선: KVPress 확장 / 백업: Cache 직접 제어

Pipeline 코드 리뷰

- 로그·질문이 모델 입력으로 들어가는 전체 흐름 확인
- 긴 입력을 먼저 처리해 KV Cache가 생성되는 단계 확인
- Press가 추론의 어느 시점에 개입하는지 파악
- 압축된 Cache가 답변 생성에 쓰이는 흐름 분석

백업 경로는 어디까지?

- KVPress가 막힐 경우 Cache 직접 제어 검토
- layer 별 Key/Value가 Cache에 저장되는 구조 확인
- 새 token 생성 시 Cache가 늘어나는 방식 확인
- token 일부만 남기는 직접 slicing/crop 방향 검토

왜 같이 분석하나?

- 본선 구현이 막혔을 때 빠르게 전환하려고
- 외부 로그 중요도 정보를 KV에 직접 적용 가능한지 판단
- PoC 단계 구현 리스크·시간 손실을 줄이려고
- 본선/백업 중 어느 경로가 현실적인지 비교

각 PRESS 의 간단한 역할

토큰을 어떤 기준으로 남길지 — 방식 비교

StreamingLLMPress

- 의미보다 **위치** 기준으로 판단
- 처음·최근 token 우선 보존
- 단순 baseline 으로 좋음

ObservedAttentionPress

- 실제로 많이 참고한 token 중시
- Attention 흐름으로 중요도 계산
- 모델 내부 신호 기반

SnapKVPress

- 최근 token 이 과거를 얼마나 보는지 관찰
- 최근 문맥 기준 과거 KV 중요도 추정
- window 기반 압축

PyramidKVPress

- layer 마다 남기는 token 수 다르게
- 낮은 layer 많이 · 높은 layer 적게
- adaptive budget 방식

우리 포인트 · 기존 Press 는 위치·Attention·최근문맥 기준. **LogPress** 는 여기에 **로그 line 중요도 신호**를 추가하려는 방향

5. 이번 주 결과와 다음 주 계획

이번 주까지 한 일

완료

- Transformer 내부 token 구조 변화 학습
- KV Cache 가 생성·증가하는 과정 이해
- KVPress 구성 요소별 역할 정리
- Pipeline 에서 압축이 들어갈 위치 분석 중

다음 주 계획

예정

- KVPress 백업 경로를 더 조사
- Tensor·module 개념 추가 공부
- Pipeline 코드 리뷰 더 진행
- 본선/백업 중 PoC 경로 판단

그 다음 목표

이후

- Thunderbird 데이터셋 사용
- 로그 line 중요도 기준을 실패데이터에서 검토
- LogPress 구조 설계
- 기존 압축 방식과 비교할 실험 방향 정리

“이번 주엔 KVPress·Pipeline 구조 이해에 집중했고, 다음 주엔 백업 경로·Tensor/Module·Pipeline 리뷰를 보완한 뒤 Thunderbird 기반 LogPress 설계를 고민할 예정입니다.”

PART D

주간 진행 보고

킬스위치 관문(M1) GO 통과 · Nezha 2층 데이터 착수 · 다음 단계 설계

한 주 한눈에

이번 주 타임라인 — 관문 통과에서 2층 착수까지

화

08/04

AUC 매칭 코드 구현

킬스위치

수

08/05

위치 정당화 → GO 선언·서명

M1 GO

목

08/06

다음 플랜 · Nezha 그룹핑 검토

고민중

금

08/07

Nezha 날짜별 확정 · 세일링 포인트

방향 확정

이번 주 핵심 1층 안전판(킬스위치) 관문 통과로 발표 가능 최소선 확보 → 곧바로 2층 RCA delta용 Nezha 데이터로 무게중심 이동

화·수 — 킬스위치 관문

1층 안전판 관문(M1) — AUC 검증 → GO

화 08/04 · AUC 매칭

음성(정상) 표본 매칭 파이프라인 구현 (1:1 / 1:4)

1:1 매칭

0.3508

1:4 매칭

0.3491

1:1 = 교란 제거 · 1:4 = 음성 ↑ 강건성 · 두 값 근접 → 1:1도 충분히 강건

수 08/05 · 정당화·GO·서명

M1 GO

- 위치 = 라벨 누수 없는 유일 지표 → GO 근거
- M1 킬스위치 GO 선언
- 버전핀 · 프리즈 합의 서명

왜 위치 단독으로? · 장애의 시간적 군집성(burst)은 인과의 본질 → 로그 일반의 성질. 썬더버드 실측 원인 평균 위치 **0.361** vs 정상 **0.501** — 앞쪽 쏠림 (일반화는 향후 검증 과제)

목·금 — 2층 데이터 & 방향

Nezha 데이터셋 착수 — 그룹핑 확정·세일링포인트

목 08/06 · 검토

검토

- 다음 단계 플랜 수립
- Nezha 데이터셋 그룹핑 방식 검토 (고민 단계)

금 08/07 · 확정

확정

- Nezha 그룹핑: **날짜별로만** 묶기로 확정
- 완벽한 구조화보다 ‘보기 편한 정도’ 우선
- 세일링포인트 재확정

왜 Nezha 인가 · 2층 RCA delta 를 검증할 **유일한 원인(root-cause) 라벨셋**. Thunderbird/BGL 은 ‘이상’ 라벨이라 진단 검증 불가

다음 주로 — 실행 항목

1

보존율 채점기

정답지(라벨) 구축 + 채점 방법 설계

2

RCA 객관식 문제

AI에게 낼 제약형(K지선다) 문항 제작

3

실험 계획서

ablation 설계 문서화

공통 참고 GPU/CPU 필요성 — 별도 컴퓨팅 환경 스펙으로 정리 | B의 Nezha 처리 주간 점검 (상시·저순위)